

Claude code入门与GitHub协作开发

Claude code有什么用

trae, Claude code, *codex*

- 完整的理解和处理你的整个项目：它可以完整的阅读你的项目代码和操作修改你的代码
- 核心设计claude.md：这个设计文件在每次对话开始就加载，你的项目背景目的，你自己的风格习惯，这些都会被写入，不用你再告诉他，相当于不用从头培养员工，它一招进来就是公司的老员工
- 更深入的交互体验：Claude code是终端对话操作的，相比于云端沙箱，它更注重与人的协作开发，不是我给你下任务，我看执行怎么样，在使用Claude code过程中，你会自动深入的了解你的项目提高你自身的能力
- 更适合复杂项目的处理：因为Claude code的执行速度偏慢，它会多从你这里讨论获取信息，对更复杂的任务完成度要更亮眼

一、Claude Code安装（windows电脑为例）与进阶使用教程

安装导航：

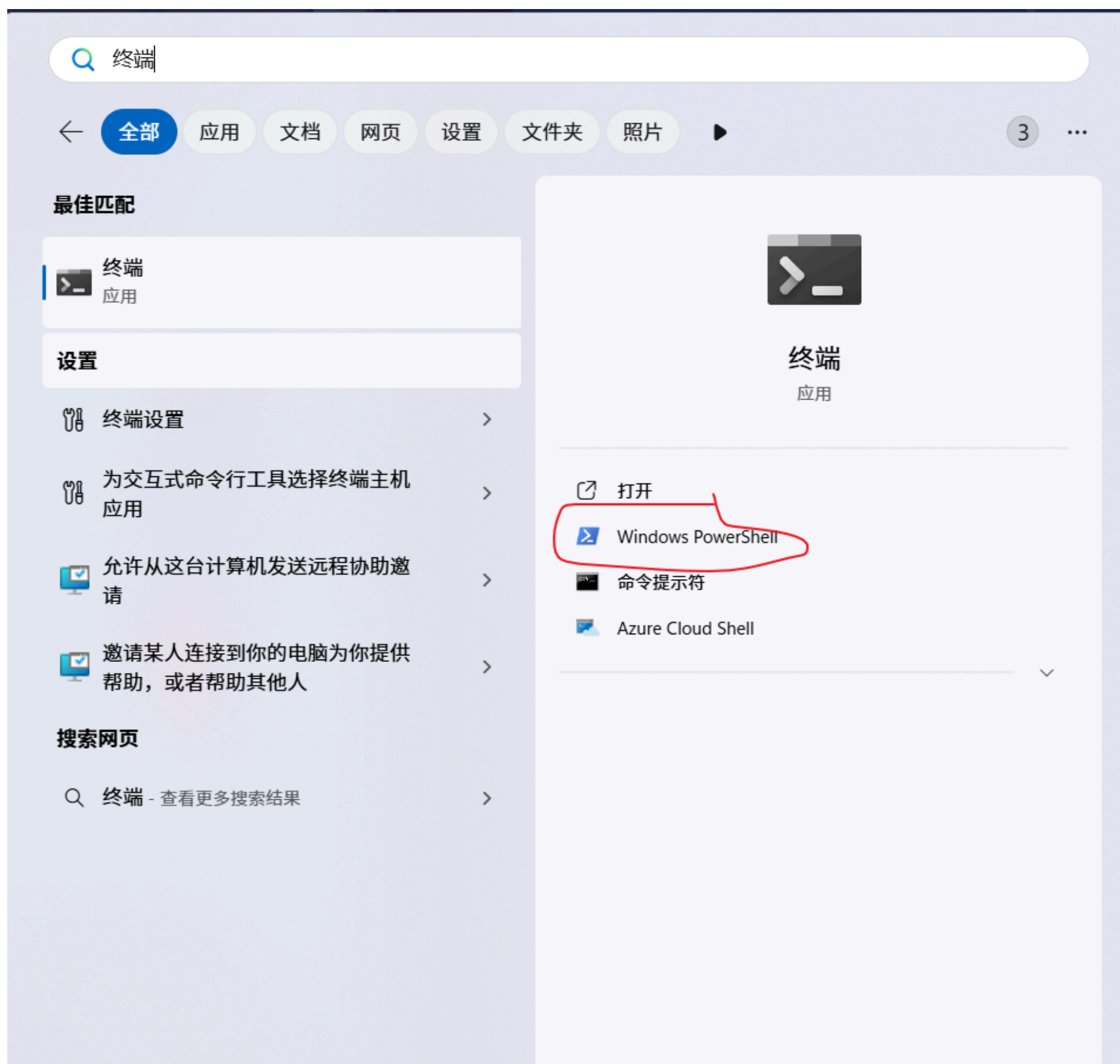
1.安装git（这里是仅windows用户需要）

git的官网是：<https://git-scm.com/install/windows>

2.安装Claude code 本体

进入终端：

注意是powershell不是cmd（一个是PS C:\Users\Lenovo>一个是C:\Users\Lenovo>）



(1) 如果你有梯子的话（下面时官网命令）：

代码块

```
1 irm https://claude.ai/install.ps1 | iex
```

有时候可能会有无法访问claude的服务器（因为anthropic对中国的ip防御太强），
因为即使我们开启了魔法，但是很多时候只有浏览器默认走的是国外流量，但是终端这些还是会走国内网络，
那么我们就开启我们梯子的虚拟网卡模式，下面以我的作为例子：



开启这个模式后我们的所有流量都会走国外的流量！
再去终端粘贴我们的命令试试！

或者会遇到这种情况：

```
irm : Just a moment...*{box-sizing:border-box;margin:0;padding:0}html{line-height:1.15;-webkit-text-size-adjust:100%;color:#313131;font-family:system-ui,-apple-system,BlinkMacSystemFont,"Segoe UI",Roboto,"Helvetica Neue",Arial,"Noto Sans",sans-serif,"Apple Color Emoji","Segoe UI Emoji","Segoe UI Symbol","Noto Color Emoji"}body{display:flex;flex-direction:column;height:100vh;min-height:100vh}.main-content{margin:8rem auto;padding-left:1.5rem;max-width:60rem}@media (width E
nable JavaScript and cookies to continue(function(){window._cf_chl_opt = {cFPWv: 'b',cH:
'drRZZeFTd5YSbf26dSrHlmcG_wkN5
b0FBz1i5yUHzNk-1778855073-1.2.1.1-
UAMbNGYPdTb5J9tgu5TREukJpeA.JP97HOHsGb.ZTrNCiL8OpoCwXE7CavOcSgBD',cITimeS: '177885507
3',cN: 'n0yXmzJLT0c67v2aVeCRD0',cRay: '9fc2ce90ec54816b',cTplB: '0',cTplC:0,cTplO:0,cTplV:5,cType:
'managed',cUPMDTk:"/
install.ps1?__cf_chl_tk=fgETwdDYjZaryUnwzkCLgo3WteU8vt_qSsLrxxOf0dg-1778855073-1.0.1.1-
LANjZ0tqgz4o1jQtOxEHb7YegkQiuvo
S4LnKi5VGw4",cvld: '3',cZone: 'claude.ai',fa:"/install.ps1?
__cf_chl_f_tk=fgETwdDYjZaryUnwzkCLgo3WteU8vt_qSsLrxxOf0dg-17
78855073-1.0.1.1-LANjZ0tqgz4o1jQtOxEHb7YegkQiuvoS4LnKi5VGw4",md:
'6i_gbQSXw8SmkELbPqgyY6N_kgghCxCRlXzTS7L3xCc-17788550
73-1.2.1.1-
uTrrK4vKRIRgZofKdSglTDvH.1JbTwpzd4bCLZbSrC39Q0uJeYNQyXtpdqRc0Mn2OA3gd.tX8E9Nc9SN73MjJNONYVLhxxRGEp94
Qyee2hbA
p6C_Ejtw3_S2RSoyEs_b2ro_ZhqjHQMZgls5jM52wfl_3d9U7vtmoySV7BLbir.ChohNHmzzyAYWIGyx_3VN9pUHF8SCsPmDpsx4
EAZJZfluTeSOOO58.xQ
```

YiqiHAFZgVgbnYcQ8J_sAlNCajy0dCQRA.cReTgwz7pltbWiHZZYpr4z.Slwn7y1P4Qw_65bj3X2ItkGHtOW06oUtaQb1ETwbvh.NZ
RvU.ivF_t6xhcKym

fp4KXq04r1JtnJY3URvYAJRhfpbFlgnGMGFvdj7lG_ArjnsMCHS1lsgilVoaGPNlNF7XFVNiHJDz9VENHSje2S9i3n7YTymPud5uQ8bf
bHetDlpOdn46a44

SsHQQzbX3gOdb0.Bq1pgV2KU0UCeUmbf2NHJJlXtIFXdU7Ojkk6Eais81DFX6hsaSZY4AaeWdJ9S7ae_gxLsLdXRWvn1_jucnbe
QofLwfoeQukV3KwxoTC

d..AVlmeOhvVSn1.yEHhGxjK_RPhoO7oFi48JnUisToPaF8FKwDwV9IzENxqTzVOnr9pV9CcfdXj6Rlv3vESCSWChZSV_nVNmuS9m
e9BdK6FtNCugS59Z4U

fooZM1yQOJ5EGthxZEnws3ny7H.ivnUXinvcxMnhY4.dYDWyAvC9hAs8yVsUQTde_i5Nxx_KCE_Ks.x9tze2SUKCPjZL0N6Nkz0WE
6YNZ6QavkE_rUnScuN

3AMuN40f4.m1KbOdiultJ7i9HuwaYYICwqIWmBQeLoQ4Mek1oxE1hRELuto6LsuFecjo8Hd0MFVilhFt1KMW7IsXMHV_k_3qQwJn
JUGR8GAzLqGcOo1Zhmcl

JHlfVFDP0lKd4kYmflg4hvDuO7SLg9_Lhc_8jCu1H2pHXIElhPq6hswP.Qd6AFVbQijbmrB6y5aZ21.dVlePG.NSFfroEkVRh48rhcaM
Q',mdrd: 'XWtd7

11vivA6CNqA3YLR6MYZxCVmEBemmDfWwUs5u4s-1778855073-1.2.1.1-
5JePhrAybcfNZdze3CnV3ADRHytAxkyDeaiLbp57pn_.ibbJAefc1W1EeZeW

jaqMxUEOVPnHdV.mEyOLIsjhP.wjLvvlvElFoY4HeV0zId_dBGy6LelvHX21pxeRQMlgJOxrUP3zfCqSd_VilZcvj8HW.4DJtiMF.16W8y
NTKdmU.y.wlJS

YVx6dCKcMPsOcExQab9eUhpNv6zN3tOC7zWLP_RU3LiquHjUuHPKpMTODzXXLu6QvPURwcjm_A9vATsm679Wht79qYnF3Rs9
.hBFVbCz4Ums5b3l37v16o

',};var a = document.createElement('script');a.nonce = 'n0yXmzJLT0c67v2aVeCRD0';a.src = '/cdn-cgi/challenge-platform/h/
b/orchestrate/chl_page/v1?ray=9fc2ce90ec54816b';window._cf_chl_opt.cOgUHash = location.hash === '' &&
location.href.ind

exOf('#') !== -1 ? '#' : location.hash;window._cf_chl_opt.cOgUQuery = location.search === '' && location.href.slice(0,
location.href.length - window._cf_chl_opt.cOgUHash.length).indexOf('?') !== -1 ? '?' : location.search;if (window.histo
ry && window.history.replaceState) {var ogU = location.pathname + window._cf_chl_opt.cOgUQuery +
window._cf_chl_opt.cOg

UHash;history.replaceState(null, null,"/install.ps1?__cf_chl_rt_tk=fgETwdDYjZaryUnwzkCLgo3WteU8vt_qSsLrxxOf0dg-
17788550

73-1.0.1.1-LANjZ0tqgz4o1jQtOxEHb7YegkQiuVOS4LnKi5VGw4"+ window._cf_chl_opt.cOgUHash);a.onload = function()
{history.re

placeState(null, null, ogU);}}document.getElementsByTagName('head')[0].appendChild(a);})();

所在位置 行:1 字符: 1

+ CategoryInfo : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExce
ption
+ FullyQualifiedErrorId :

WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

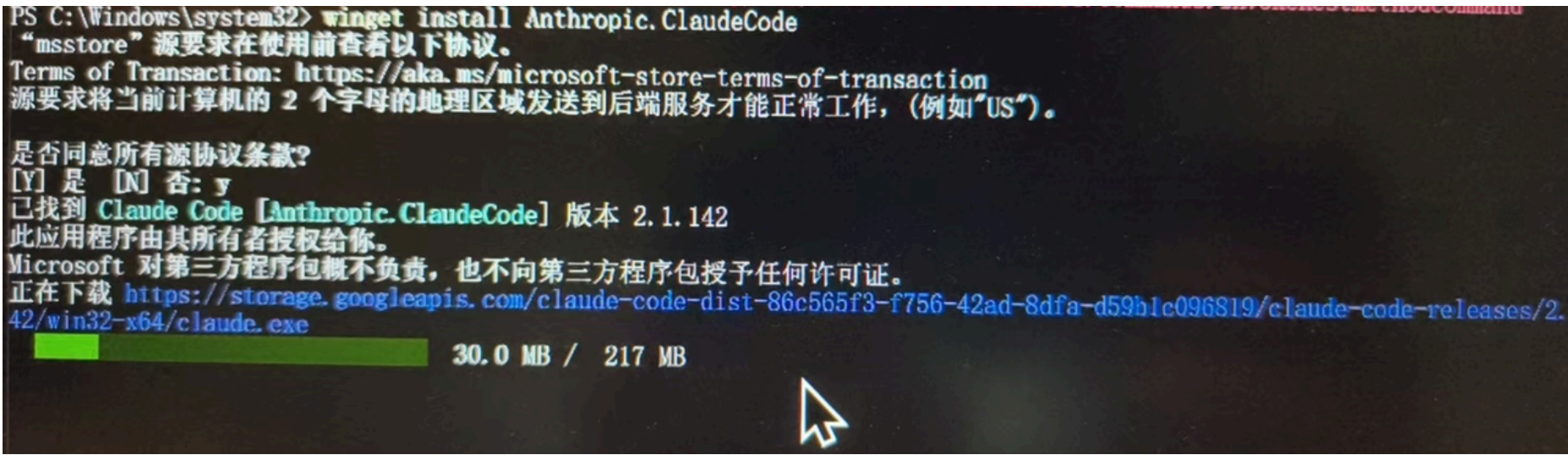
这行一大坨错误命令意思是我们的请求被cloudfare这家网安公司拦截了，如果这个类型的命令不行我们换一下这种：

代码块

```
1 winget install Anthropic.ClaudeCode
```

再去终端粘贴我们的命令试试！

如果是这样的，基本就是正确的：



(2) 如果你没有梯子的话，而且上面的方法也没用，那么试试这个博主的镜像网站（终极大招）：

irm <https://daheiai.com/cc.ps1> | iex

运行后，会向这个博主发送请求，由他的服务器继续向anthropic发出请求从而在你的电脑上下载（对安全有担心的可以暂时不用）

3.配置环境变量（注：winget会自动配置好，直接跳过这一步，irm方案的需要进行这一步）

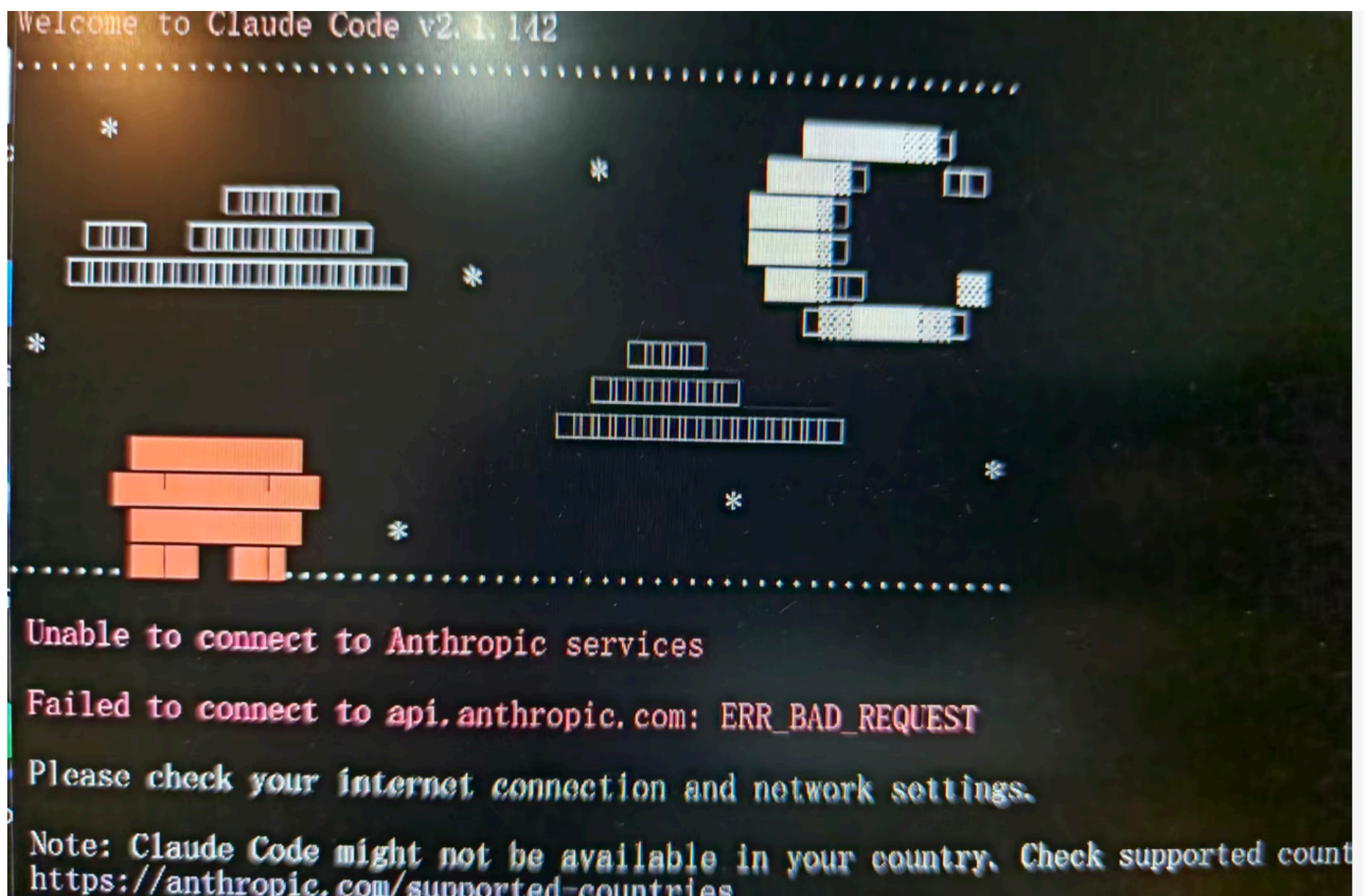
点开开始菜单，输入环境变量，点“编辑系统环境变量”。

在窗口下面点“环境变量”，在系统环境变量里找到 Path，双击它。

新建一条，把 刚才安装成功显示的Claude的文件路径复制进去。

4.安装ccswitch软件，接入更多模型

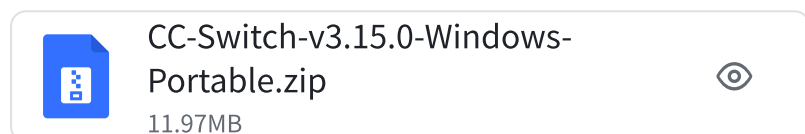
刚才我们安装好了Claude code，他会让你去终端再启动一遍，然后输入“Claude”运行，你会发现：



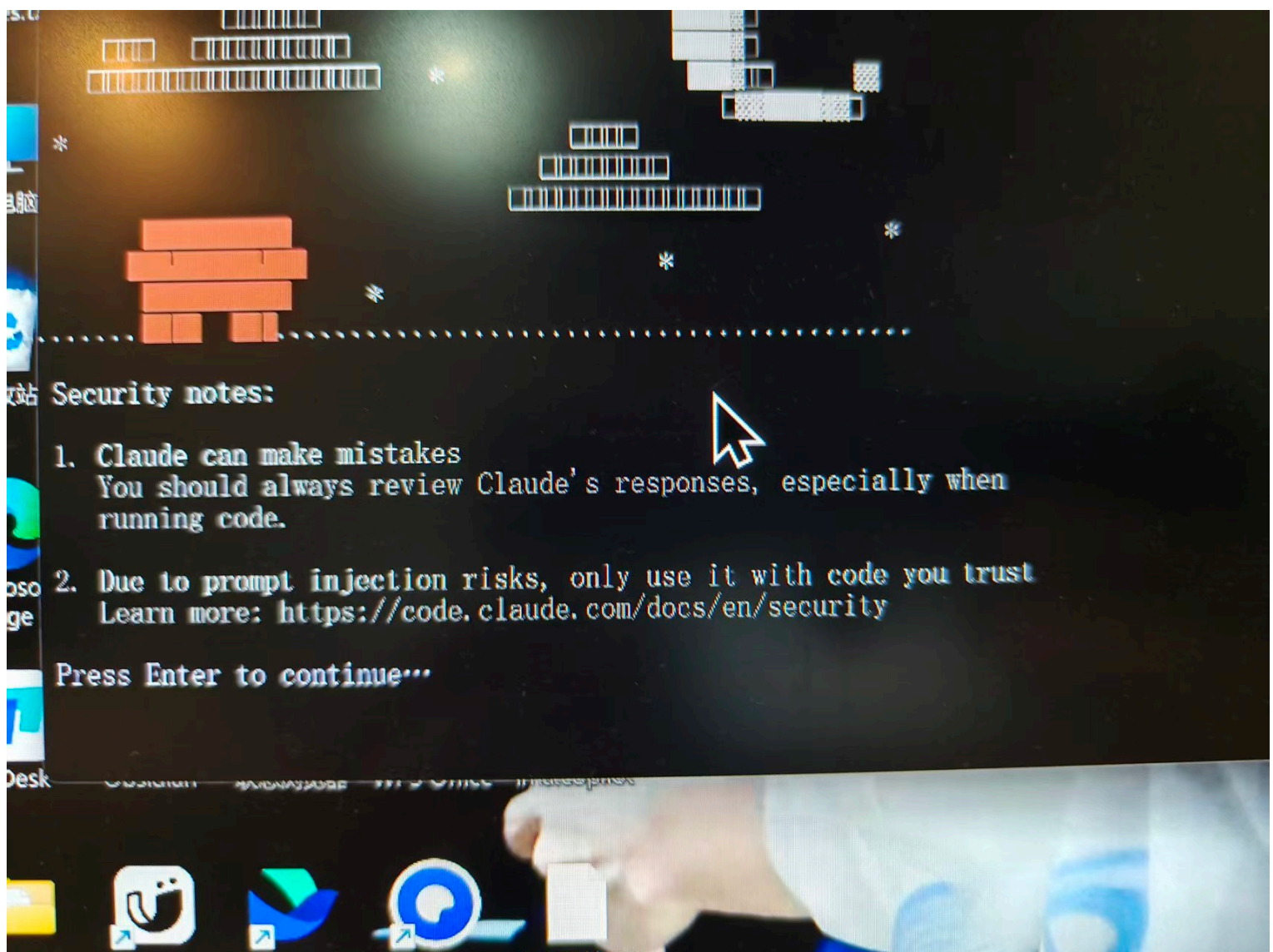
也就是Claude拒绝你的访问，我们安装下面的软件，把请求地址换成我们国内的大模型：

<https://wwayc.lanzoub.com/iiHKV3n29yhe>

点击这个链接下载cc-switch，然后配置模型，这里建议deepseek



配置好后我们就会发现，他不会再向anthropic请求，就会开始“Claude也会犯错”呀，这些就可以使用了



接下来打开任意一个你需要工作项目的文件夹，右键在终端中打开，输入"claude"就可以开始与Claude code一起写作了！

二、Claude code 的一些小玩法（帮助你快速上手）

1. 常见的plugin命令

我们首先说说：plugin是什么，你就把他了解为我们的插件或者skills，这些东西是已经被封装好的技能包，只要你运行plugin就可以触发它

- /init

用于项目初始化，如果你有了自己雏形的代码，可以让Claude code（以下简称CC）快速理解你的项目并生成claude.md（这个文件会在每次加载上下文时运行）

- /clear

用于清除上下文，重新开启一个对话，注意他会清空你开启窗口后你与cc的所有对话，也就是说下一次你将于一个什么都不知道的cc对话，常用的做法是提前让cc总结你与他对话的精华，放到下一次对话输出给他

- /compact

用于让cc自动帮你精简上下文，初始上手对编程不熟悉可以运行，但是开发到成熟的后期，不建议让cc自动压缩上下文，因为他会自己判断，把一些无用的报错和污染的上下文精简进去

- /cost

查看当前上下文占比，你最高频的指令，因为到百分之60的上下文占比，cc会开始变蠢，记得提前压缩上下文

- /help

查看当前可用的所有命令

- /plan

顾名思义，这个指令是开启计划模式，在这个模式下CC只问答不执行，在拿不准方案或者追求准确输出的时候可以开启与cc反复确认你的项目路径和规划

也可以通过“shift+tab”来在Default → Auto-accept edits → Plan这三个模式内来回切换

2. 个人比较推荐的skills安装（新手开箱即用）

skills是什么呢：本质上是一个markdown文件，它包含触发条件和工作流程，是一个可复用的技能包，在一些特定领域的工作上比如前端设计，文档处理等特定领域触发，非全局

- supperpowers

包含什么：/brainstorm — 苏格拉底式提问，帮你把模糊的想法打磨清楚，/write-plan — 把设计拆成2-5分钟的具体小任务，/execute-plan — 按计划逐步执行，不会乱跑

为什么装：如果刚接触一个项目，不知道怎么去规划，怎么去执行，怕跑偏就可以用这个帮你快速先建立你的项目骨架

常用场景：用/brainstorm去让cc问清楚你的项目要做什么，要运用在什么场景，需要做什么，先问清楚，接下来的两个核心技能不用手动触发，因为在执行任务时基本都会自动遵循这个原则去执行

- Skill Creator（A社官方）

可以做什么：通过问答引导你，自动生成完整的skills文档；帮你优化已经有的skills文档

为什么装：如果你发现你自己在工作项目时总有一些常见的要求，或者固定的要求cc给你输出的要求，你可以用这个快速创建一个符合你自己习惯的skills

- frontend-design

可以做什么：可以帮你产出脱离“AI平均值”的前端网站设计；适配Vue、React等主流框架；产出有设计感的前端组件

为什么装：AI生成的网站如果个人没有很好的创意或者描述不清楚，就难免走向AI平均值，走千篇一律的风格，这个可以较好的解决这个问题

这里需要说明白的是Claude code的迭代速度非常之快，没有指定的命令可以稳定运行，所以想安装这几个skills最好的方案就是直接去问你的Claude code，让他帮你安装

开源社区现在有非常丰富的skills，我们可以自行去GitHub等开源社区看有没有适合自己的

3. 安装前必读的三条规则

第一：先装少，用够再加

Skills的数量不是越多越好。CC会预加载所有Skills的名称和描述，装太多会让CC在"选哪个"这一步就出错。

建议起步只装 Superpowers + Skill Creator，其他等有真实需求再装。

第二：按项目装，不要全局装

项目级安装（只在当前项目生效）：

代码块

```
1 /plugin install xxx
```

全局安装（所有项目都能用）：

代码块

```
1 /plugin install --global xxx
```

不同项目有不同需求，分开装比全部塞进全局干净得多。

第三：用斜杠命令100%触发

Skills的自动触发不稳定，尤其是用非Claude原生模型时。

需要某个Skill时，直接输入斜杠加Skill名称强制触发，比等它自己判断可靠：

代码块

```
1 /brainstorm
2 /write-plan
```

三、操作习惯

没有概念都是坑，提前知道，用的更顺畅！

1. ESC键：单按 vs 双按

这是最容易搞错的操作，后果差别很大。

单按 ESC — 中断当前任务，CC停下来等你说话，上下文还在

双按 ESC — 直接退出CC，相当于关闭终端，上下文全部丢失

2. 上下文管理：什么时候该开新对话

用 `/cost` 查看当前上下文占比。**到60%左右就该处理了**，超过这个阈值CC会开始变蠢，出现幻觉，忘记前面说过的事。

正确的流程：

第一步，在当前对话里发这句话，让CC整理进度：

代码块

```
1 帮我总结当前对话的决策日志：
2  【已确定的技术决策】
3  【已完成的模块】
4  【当前正在做的任务】
5  【未解决的问题】
6  【下一步计划】
```

第二步，自己读一遍，看有没有遗漏或说错的，手动补充修正。

第三步， `/clear` 清掉当前对话。

第四步，新对话第一条消息把这份总结粘贴进去，CC就能在正确的上下文上继续工作。

本质：你来做记忆，不依赖CC自己记。

3. 权限模式：给CC多大的自由度

CC默认每次操作都会来问你批不批准，安全但很打断思路。你可以用 `Shift+Tab` 在几个模式之间切换，状态栏会显示当前模式。

四个常用模式：

默认模式（Default）— 每次操作都询问你，新手推荐，知道CC在干什么

自动接受模式（acceptEdits）— 文件修改自动批准，只有shell命令还需要确认，按 `Shift+Tab` 切换

计划模式（Plan）— CC只读不动，只分析不修改文件，用来评估方案、理清思路

全权放行模式（`--dangerously-skip-permissions`）— 跳过所有权限确认，CC自己全跑完（注意这个不在shift+tab的切换选项中）

关于全权放行模式，单独说清楚：谨慎使用

4. @引用：让CC只看你指定的文件

在对话里输入 `@文件名` 可以直接引用项目里的某个文件。

代码块

```
1 @App.vue 这个组件的提交逻辑有问题，帮我检查一下
```

这样CC只会去看这一个文件，而不是扫描整个代码库，节省上下文，回答也更精准。

适合场景：你已经知道问题在哪个文件，不需要CC全局分析的时候

四、GitHub协作开发

GitHub我们都知道是现在最大的开源社区，有很多大佬的项目，当然也有很多小白和半成品项目（需要辨别）

很多时候点进去都是英文（即使做了中文优化）但是一些专业名词还是看不太懂，但是我们不需要读懂它：

我们尽量去避免和GitHub直接接触，去最大化发挥他的价值：

一：首先就是他是一个很好的”跳板工具“

为什么是一个很好的跳板呢：因为我们部署到GitHub上的项目可以直接通过它与很多代码托管平台联系，快速部署上线！

这里列举几个好用的，可以直接与GitHub联系的：

1.vercel：几乎是我个人最推荐的前端部署平台，他有很多的优势：比如自动同步GitHub的代码更新，对前端框架适配很强，可以把它作为一个专业的前端部署平台

但他有他的缺点：他会自动给你分配一个网站，但是这个网站被国内屏蔽的概率比较大，如果你有自己的国内域名就很好

2.netify：这些因为学长讲过了，不再细讲，但是单独说是因为他是现在几乎唯一一个国外托管平台自动给你生成一个域名后，对国内用户访问友好的网站

3.render：专业的后端服务平台，也是支持自动同步更新GitHub代码的变动

所以有个流程就是：

用AI写好代码———部署到GitHub上———部署到前后端服务平台上

这里注意，这个流程适用于真正的全栈开发（也就是前后端项目），如果是静态的个人网站或者展示页面，其实没必要通过这种方式去上传

也就是说，通过GitHub，你省了部署前后端服务的时间和后期修改前后端代码的时间

二、怎么去把本地的项目部署到GitHub上

情况一：全新项目第一次推上去

第一步：在 GitHub 上建仓库 点右上角 + → New repository → 填名字 → Create

第二步：本地初始化

bash

代码块

```
1  git init                # 把这个文件夹变成 git 仓库
2  git add .               # 把所有文件加入暂存区
3  git commit -m "first commit"  # 打个快照，引号里是备注
```

第三步：和 GitHub 关联并推上去

bash

代码块

```
1  git remote add origin 你的仓库地址  # 告诉本地，远端在哪
2  git push -u origin main              # 推上去
```

情况二：之后每次改完代码更新

每次基本都这么用：

bash

代码块

```
1  git add .
2  git commit -m "改了什么"
```

```
3 git push
```

报错情况一：网络连接不上

还是前面我们解决cc下载的思路，开启虚拟网卡：

或者感兴趣的可以试试下面这种方案，在终端映射你的端口，

代码块

```
1 git config --global http.proxy http://127.0.0.1:7890
```

代码块

```
1 git config --global https.proxy http://127.0.0.1:7890
```

不想用了就取消

代码块

```
1 git config --global --unset http.proxy
```

更新报错情况：远端版本高于本地

一般说是这样：![rejected] main -> main (fetch first)

这就是远端仓库的版本高于本地了

我们可以这样做：

先 pull 再 push

代码块

```
1 git pull origin main # 先把远端拉下来合并
2 git push             # 再推上去
```

三、怎么去试着逛GitHub

除非你是特意想去研究一些项目的文件结构或者深入学习开源项目，大部分情况下GitHub上的项目是充当一个“AI素材库”的角色，不用我们每天都要去看

所以我们分一下几步走：

先列整体思路：

描述你的项目需求（喂 claude.md）



大模型推荐相关项目 + Awesome 系列辅助



快速筛选：Stars趋势 + 最近commit时间（先排除死项目）



AI 分析：README / CHANGELOG / 依赖文件



人工精读：Issue（聚焦 bug/wontfix/你关心的场景）



第一步：找到与你项目强相关的项目

就比如我个人的微信机器人项目，我要怎么找到和他强相关的？

首先就是你可以把你自己的项目信息和背景目标告诉大模型（这里我个人推荐Claude），如果你已经下载了CC，那么你可以把 claude.md喂给模型，然后由它替你逛，帮你找到合适的项目

第二步：筛选

我们在这一步判断什么，欢迎程度和是否为活项目

star很简单，和免费的关注红心没区别，只不过他的含金量要高很多，大部分情况可以作为质量的参考

判断是死活项目呢，这里我们不用自己去看，因为评估一个项目的情况很多，我们让AI来帮我们判断：

我们摘取这个项目的链接，直接发送给AI，然后配上prompt：

“请作为高级架构师，对这个 GitHub 项目 [链接] 进行‘健康度审计’。重点检索并分析：

1. **更新频率**：过去 6 个月内是否有实质性的代码提交（Commits）？
2. **版本历史**：最近一次 Release 是什么时候？是否兼容当前的 Python/Node.js 主流版本？
3. **社区响应**：最近 10 个 Issue 的平均回复周期是多少？是否存在大量积压的 Bug 类标签？
4. **维护风险**：它是单人维护还是组织维护？ **最终结论**：请直接告诉我这个项目是‘处于活跃期’、‘进入维护期’还是‘已经脑死亡’。”

第三步：人工看issue

issue是什么：

这个issue其实就是抖音这些视频平台的评论区，就是我们刷视频刷到了，觉得不错欸，可以留下个评论，问个问题

但是这个要带入到程序员的视角，他就会偏向于纯粹的技术交流，不再是评论那种混合的，十分纯粹

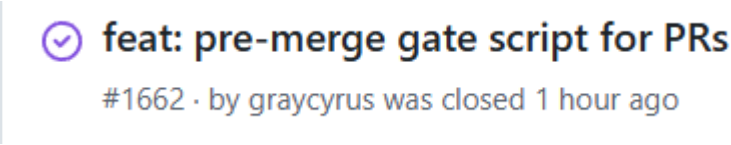
一般是有人用了作者的项目后，给用户的反馈，一般就是报错和改进建议

我们看什么：

在看之前先了解一下“状态”和“标签”的概念，
状态也就是“open”和“closed”这两个，他们两个区分的是这个问题（也就是评论）是否结束的问题，
标签也就是，“feature” “bug” 这些会在问题的右边标出上的，代表的是这个问题的本质属性，也就是他究竟在问什么

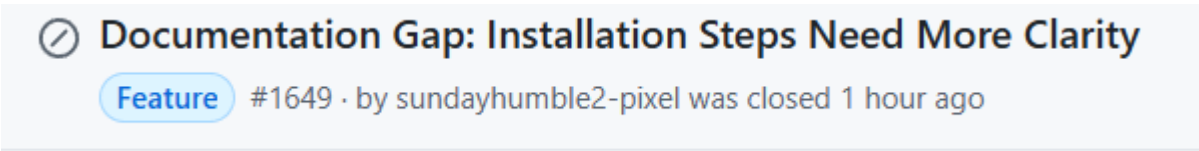
我们就不看别的，只用看“open”右边的“closed”这一部分，这里的问题一般分为两种情况，以及问题区中被打上bug标签的问题

一个是：紫色的圆圈打上了绿色的对勾，也叫：Closed as completed (已完成关闭)



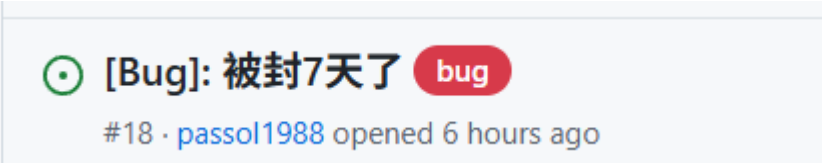
这种一版代表这个 Issue 得到了实质性的解决

另一个是：灰色的源泉打上了斜杠，也叫：Closed as not planned (未计划关闭 / 拒绝处理)



这种就意味着它的问题依然存在，但维护者决定“结案”不修了，当然包括很多场景，这点可以继续深入了解

最后一个就是：bug



这种需要注意，代表系统出现了明确的错误

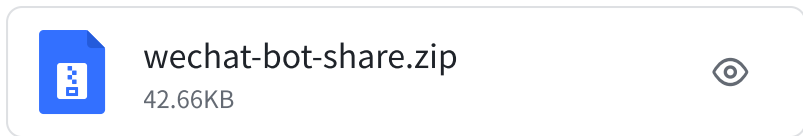
以下是常见的“标签”分类：

标签名称	核心含义	评估价值
wontfix	不予修复。 维护者承认这是一个问题或需求，但出于架构限制、历史包袱或与项目核心理念冲突，决定永远不予解决。	极高。 这是技术选型时的重点排雷区。如果你关心的核心场景被打上了此标签，直接放弃该项目。
duplicate	重复反馈。 代表社区中已经有人提过完全相同或高度类似的问题。	顺着维护者提供的链接，跳转到主 Issue 追踪进度即可。
invalid	无效反馈。 通常因为提问者没有提供复现代码、环境信息，或者该问题根本不是由于本仓库代码引起的（例如用户自己的环境配置错误）。	侧面反映维护者对 Issue 质量有要求。

标签名称	核心含义	适用场景
help wanted	需要社区协助。 维护者认可该需求或 Bug 的价值，但当前团队没有时间精力去写代码，呼吁社区的外部开发者提交 PR (Pull Request) 来解决。	评估该问题近期被修复的概率较低；也是你参与该项目核心贡献的切入点。
good first issue	适合新手的首次贡献。 难度极低、边界清晰且逻辑简单的任务。这是维护者专门为了培养社区新血液而预留的“新手村”任务。	快速熟悉一个复杂开源项目架构及代码提交流程的绝佳入口。
question	纯粹的技术提问。 不涉及报错，也不要求改代码，单纯是因为文档没写清楚而导致的疑问。	-

五、 我个人的项目微信群聊机器人项目

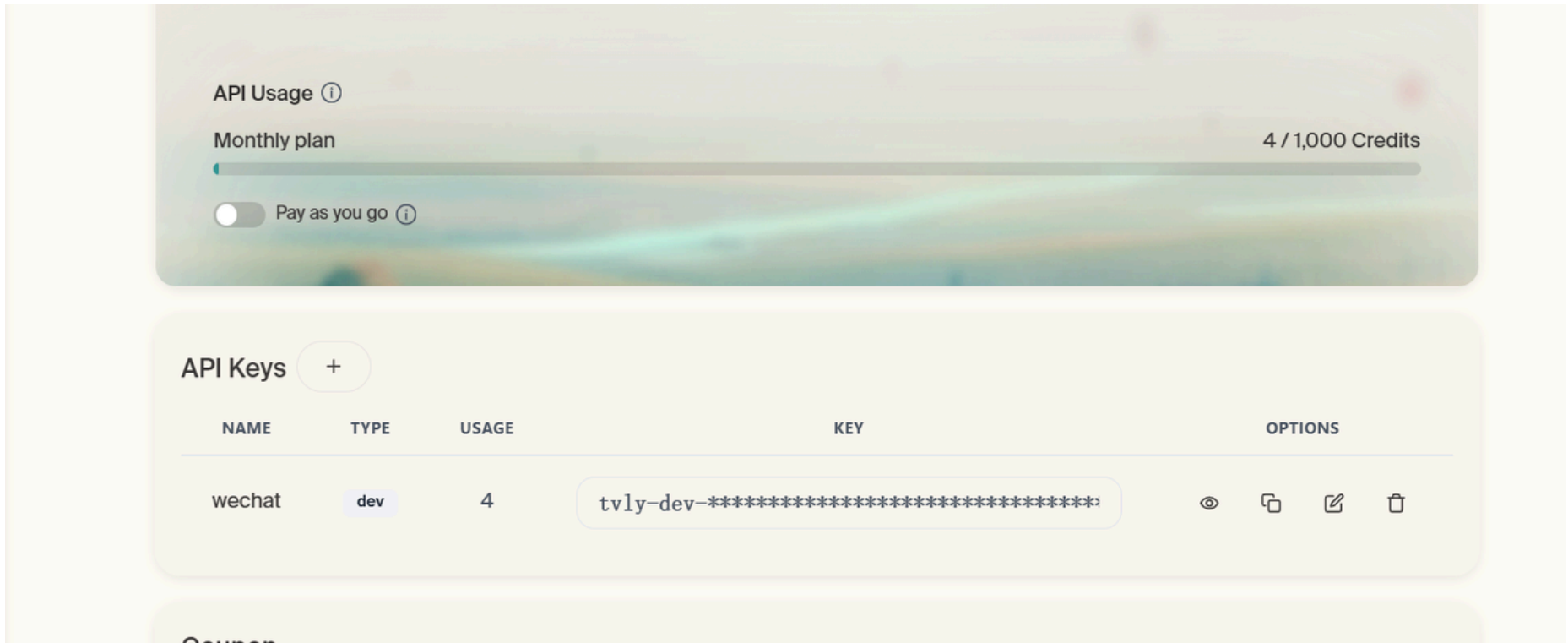
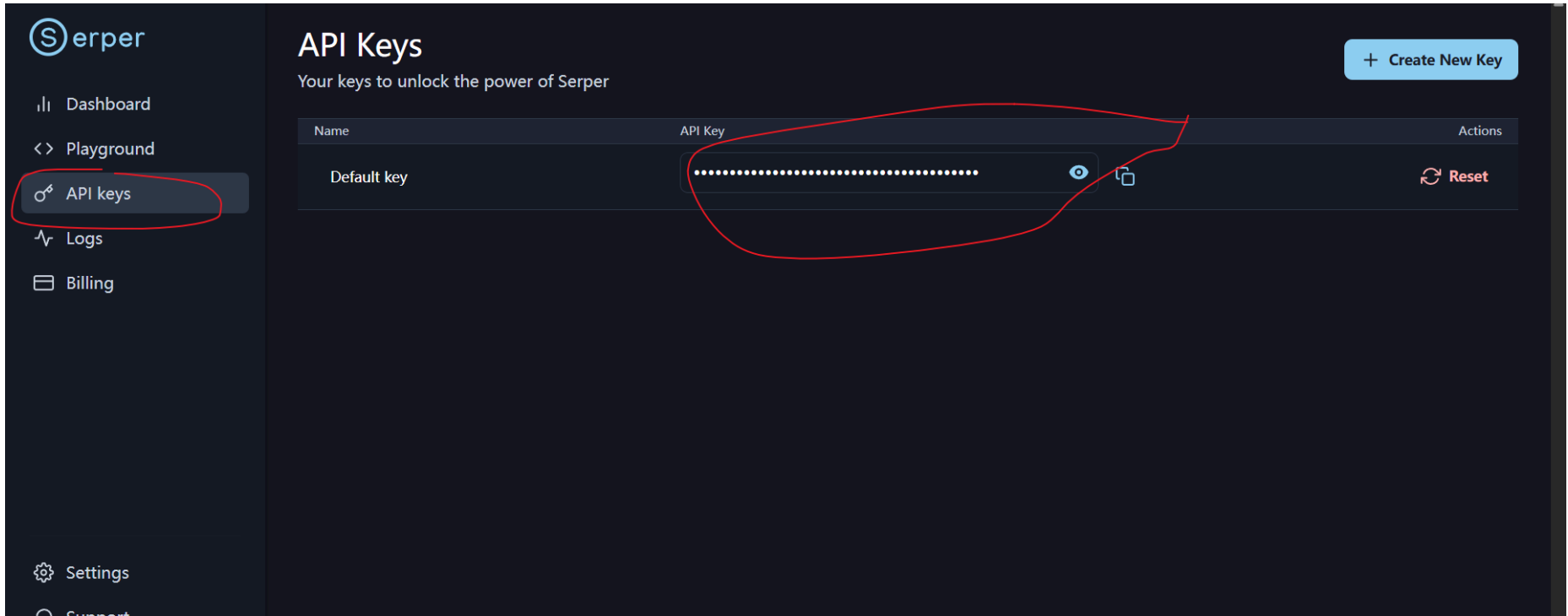
本项目的架构基于GitHub上的wx4py项目作为骨架，结合一定的延伸拓展完成



这是我个人的项目文件，先解压！

两个搜索api服务：<https://app.tavily.com/>

<https://serper.dev/>



1. 验证 Python 版本

```
py -3 --version
```

输出 Python 3.10 ~ 3.12 都行。如果只有 3.9 或更低，需要装一个新的。

确认 py -3 指向的具体路径：


```
py -3 -c "import sys; print(sys.executable)"
```

2. 安装依赖

```
py -3 -m pip install wx4py pywinauto pywin32 pillow requests
```

联网搜索（可选，不装也能用）：

```
py -3 -m pip install tavily
```

验证：

```
py -3 -c "import wx4py, pywinauto, PIL, requests; print('依赖 OK')"
```

3. 配置密钥

打开 config.py：

必需

```
DEEPSEEK_API_KEY = "sk-你的密钥"
```

可选 —— 搜索/早报，不填自动跳过

```
TAVILY_API_KEY = "tvly-你的Tavily密钥"
```

```
SERPER_API_KEY = "你的Serper密钥"
```

```
LISTEN_GROUPS = ["你要监听的群名"]
```

```
BOT_NAME = "机器人群昵称"
```

三个密钥的关系：

三个密钥只填 DeepSeek 也能正常对话。两个搜索密钥都空 → 联网搜索和早报静默跳过。任意填了一个 → 联网搜索可用。早报只认 Tavily。

4. 启动微信

- 微信 PC 客户端已登录
- 目标群聊窗口可见（不能最小化）
- 群聊需在置顶位置（wx4py 从置顶列表找窗口）

5. 启动机器人

```
py -3 main.py
```

看到  机器人启动中... 就算正常。如果没配 Tavily 会打印  未配置 Tavily API key，跳过 AI 早报调度器，这是预期行为。

<https://qumomowebste.netlify.app/>



曲墨墨



扫一扫上面的二维码图案，加我为朋友。

如果对安装cc或者想试一下我的项目，在这个过程中有疑问都可以来找我问我问题